EXPERIMENT 07

Integration with Zoho CRM using APIs

Aim

Synchronize approved library members to Zoho CRM Contacts using an authenticated connection, explicit mappings and retry-safe handling.

Procedure

1. Use a test CRM organization and a suitable contact-sync use case approved by the institution. Member circulation history is not required in CRM and must not be sent by this integration.

2. Create a Creator connection named library_crm with the CRM scopes needed for contact creation and updates, such as ZohoCRM.modules.contacts.WRITE for the upsert endpoint. Add READ only if verification or reconciliation requires it. Confirm current endpoint requirements and the authorizing user's module permissions.

3. For the lab, use Members.Email as a required unique contact matching key. Set CRM duplicate checking accordingly and document address-change handling. For production, prefer a configured unique external member key if supported and approved.

4. Map Members.Full_Name to CRM Last_Name for this simplified lab, Members.Email to Email, and the returned contact ID to CRM_Contact_ID. A real implementation should use explicit first-name and last-name fields when required.

5. Mark a saved member Pending and invoke the integration after save or through a scheduled queue. Keep the issue-book operation independent from CRM availability. Use the API domain matching the CRM account data center; the example uses the US domain.

```
// Function body: argument memberId is a Creator record ID.
m = Members[ID == memberId];
contact = Map();
contact.put("Last_Name", m.Full_Name);
contact.put("Email", m.Email);
items = List();
items.add(contact);
matchFields = List();
matchFields.add("Email");
payload = Map();
payload.put("data", items);
payload.put("duplicate_check_fields", matchFields);
response = invokeurl
[
    url :"https://www.zohoapis.com/crm/v8/Contacts/upsert"
    type :POST
    parameters:payload.toString()
    headers:{"Content-Type":"application/json"}
    connection:"library_crm"
];
```

This is the request-building portion only. Verify the member exists, validate required data, and implement response and exception handling on the next page before enabling automated retries. Never insert a raw OAuth token into the script.

EXPERIMENT 07 CONTINUED

Implementation and verification

Procedure continued

1. Inspect both HTTP failures and the CRM record-level result. The successful item is expected to contain a success status and details.id. Do not treat a nonempty response or an HTTP success alone as proof of a successful upsert.

2. After validating the response structure, store data[0].details.id as text in CRM_Contact_ID and set CRM_Sync_Status to Synced. On an exception or record-level error, set Error and record a sanitized error code and correlation ID.

3. Retry temporary failures with a bounded schedule and increasing delays. Reuse the same matching key. Route authorization failures, validation failures and repeated failures to an administrator; do not retry them endlessly.

4. Verify the stored contact ID in CRM. Repeat the same request and confirm only one contact remains. Before changing a member email, define an update-by-stored-ID path to avoid creating a second contact.

5. As a separate API exercise, read All_Loans using Creator API v2.1 with an authorized read-only token. Substitute the verified owner, app and report link names. Use a client that stores the token securely.

```
GET https://www.zohoapis.com/creator/v2.1/data/


    {owner}/{app_link}/report/All_Loans
Authorization: Zoho-oauthtoken {secure_access_token}

Example read scope: ZohoCreator.report.READ
The URL above is wrapped for print; send it as one continuous URL.
```

Apply report criteria and field selection as documented. Follow the endpoint’s documented record_cursor mechanism when the response supplies a cursor for additional records; do not assume one response is the entire dataset. Verify the current response headers and API limits in the Zoho Creator API v2.1 documentation.

Test	Expected evidence
Valid member sync	One contact; returned ID stored; status Synced.
Identical request repeated	Existing matching contact updated; no duplicate.
Missing or invalid required field	Readable error; no false Synced state.
Revoked connection or wrong scope	Authorization failure logged without secrets.
Temporary outage or throttling	Bounded retry or pending error; no extra loan.
Unauthorized Creator report read	Denied access; no member or loan data returned.

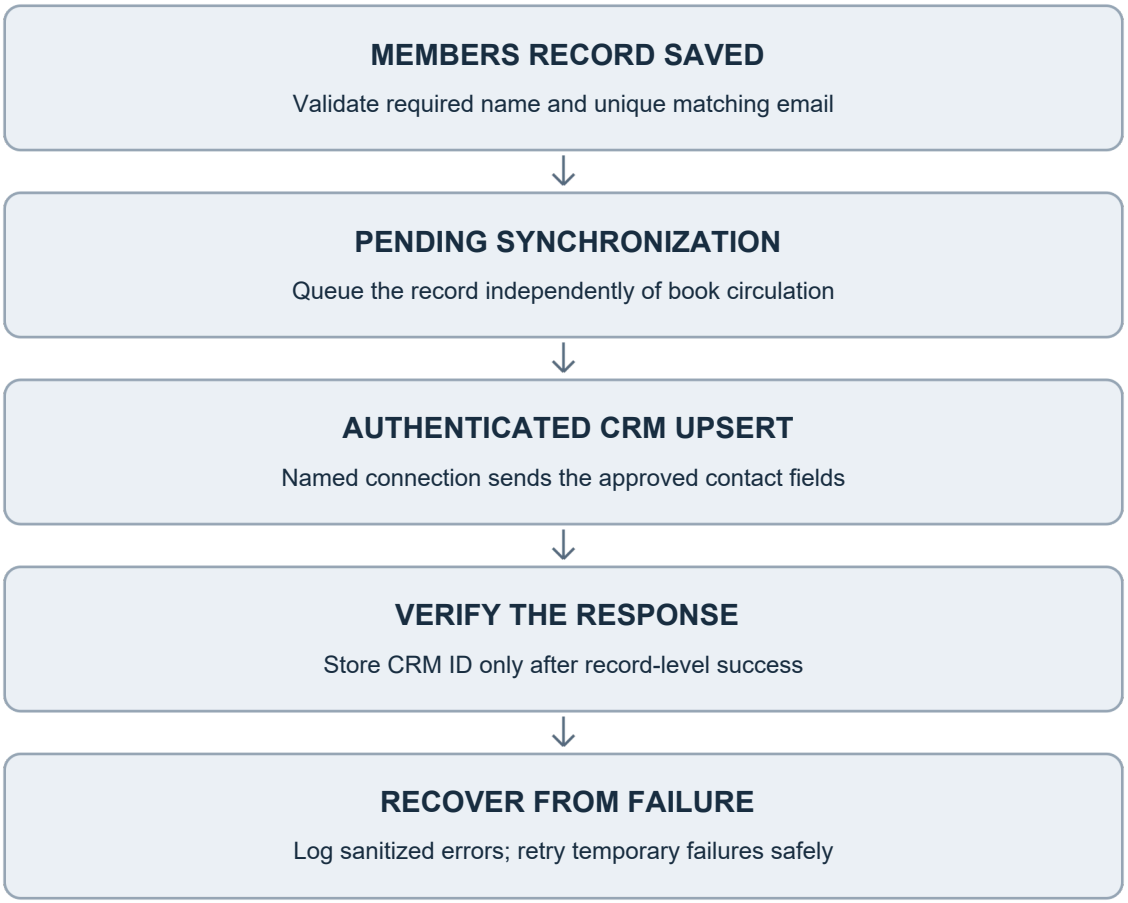
Evidence

E07: field mapping, redacted connection configuration, request shape, sanitized response, CRM verification, retry log and Creator read test. Do not capture tokens or connection secrets.

EXPERIMENT 07 VISUAL RECORD

Application images and explanation

Illustrative process diagram



Generated explanatory diagram. It is not a screenshot of an executed workflow.

A successful contact upsert should have a stored remote ID and a verified record-level success response. Use a stable matching rule, and specify what happens when an email address changes. Keep integration errors visible without automatically reversing a legitimate circulation transaction.

Result

Expected result and verification criteria. A permitted test member synchronizes once, repeat requests remain duplicate-free under the chosen key, and errors are visible and recoverable. Creator API reads respect authorization and pagination.